

**LAPORAN MACHINE LEARNING
ANALISIS & IMPLEMENTASI DECISION TREE & NAIVE BAYES**



Disusun Oleh:

Tri Haiji Januarli (G1A024069)

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026**

SOAL DAN PEMBAHASAN

BAGIAN A: ANALISIS KONSEP [20%] - Individu

Jawab secara essay 1-2 paragraf per soal

1. Jelaskan kapan Decision Tree lebih cocok digunakan dibanding Naive Bayes untuk kasus prediksi kelulusan ini. Beri 2 alasan.

Jawab:

Menurut saya, Decision Tree lebih unggul dibanding Naive Bayes pada kasus prediksi kelulusan ini jika data memiliki pola non-linear dan interaksi antar-fitur yang kompleks. Pertama, sebagai algoritma hierarkis, Decision Tree secara alami mampu memetakan kombinasi kondisi khusus seperti IPK tinggi yang tetap berisiko terlambat lulus jika Kehadirannya rendah melalui percabangan fleksibel, sedangkan Naive Bayes gagal karena mengalikan probabilitas setiap fitur secara terpisah.

Kedua, Decision Tree sangat efektif menangani campuran fitur heterogen (kontinu dan kategorikal) tanpa memerlukan asumsi distribusi statistik. Berbeda dengan Gaussian Naive Bayes yang kinerjanya rentan menurun jika variabel kontinu seperti *Jam_Belajar* berdistribusi tidak normal (*skewed*), Decision Tree mengevaluasi nilai ambang batas (*threshold*) menggunakan *Information Gain* atau *Entropy*, sehingga sifat sebaran data tidak merusak akurasi model.

2. Asumsi "independen" di Naive Bayes. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa.

Jawab:

- 1) IPK dan Jam_Belajar, karena durasi belajar berpengaruh langsung terhadap performa akademis, sehingga mahasiswa dengan jam belajar tinggi cenderung memperoleh IPK tinggi. Keduanya berkorelasi kausal dan tidak independen.
- 2) Status_Beasiswa dan Penghasilan_Ortu, karena penetapan beasiswa umumnya mengacu pada kondisi finansial keluarga, sehingga mahasiswa berpenghasilan ortu rendah berpeluang lebih besar menerima beasiswa. Kedua fitur ini saling terikat secara struktural.

3. Apa risiko overfitting pada Decision Tree di dataset ini? Bagaimana cara mengatasinya?

Jawab:

Ketika model *Decision Tree* dibiarkan tumbuh tanpa batasan (*max_depth=None*), pembelahan cabang akan terus berlangsung hingga mencapai daun murni. Hal ini membuat model rentan mengalami *overfitting* karena tidak hanya mempelajari pola umum, melainkan

juga menghafal *noise* dan keunikan spesifik pada data latih. Dampaknya, akurasi model terlihat sempurna saat pelatihan, tetapi merosot tajam ketika memprediksi data uji baru. Risiko ini kian meningkat jika fitur kategorikal seperti Organisasi atau Penghasilan_Ortu memiliki sebaran nilai yang kurang merata pada dataset.

Untuk meminimalisasi risiko *overfitting* tersebut, strategi yang dapat dilakukan meliputi:

1. Menetapkan ambang kedalaman pohon secara eksplisit dengan mengonfigurasi parameter `max_depth`.
2. Mengatur syarat minimum jumlah sampel pada simpul internal (`min_samples_split`) maupun simpul daun (`min_samples_leaf`) agar cabang tidak terbentuk pada data yang terlalu sedikit.
3. Menerapkan teknik pemangkasan (*tree pruning*) untuk memotong cabang-cabang kompleks yang kurang berkontribusi pada peningkatan performa.
4. Menggunakan *cross-validation* guna memastikan kestabilan serta daya generalisasi model pada berbagai kombinasi pembagian data.

BAGIAN B: IMPLEMENTASI & EVALUASI [50%] - Kelompok 2-3 orang

Gunakan Python + sklearn + pandas + matplotlib

1. Preprocessing: Handle missing value, encoding, train-test split 80:20

Jawab:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
```

Gambar 1.1 Mengimport library

Penjelasan:

Library pandas dan numpy digunakan untuk manipulasi serta pemrosesan struktur data, modul-modul sklearn dimanfaatkan untuk pra-pemrosesan (*label encoding*, *train-test split*), pelatihan model (*Decision Tree*, *Gaussian Naive Bayes*), serta kalkulasi metrik evaluasi; sedangkan matplotlib berfungsi untuk membuat visualisasi grafik dan *confusion matrix*.

```
# LOAD DATASET
url = "https://raw.githubusercontent.com/naylaviona/Machine-Learning/main/mahasiswa_lulus.csv"

# Membaca dataset
df = pd.read_csv(url)

print("=== CHECK MISSING VALUES SEBELUM IMPUTASI ===")
print(df.isnull().sum())
print("-" * 50)
```

```

*** === CHECK MISSING VALUES SEBELUM IMPUTASI ===
IPK                15
Kehadiran          15
Jam_Belajar        0
Organisasi         0
Penghasilan_Ortu   0
Jenis_Kelamin      0
Status_Beasiswa    0
Lulus_Tepat_Waktu  0
dtype: int64

# HANDLE MISSING VALUE
# Mengisi missing values pada kolom numerik dengan nilai median
df['IPK'] = df['IPK'].fillna(df['IPK'].median())
df['Kehadiran'] = df['Kehadiran'].fillna(df['Kehadiran'].median())

# ENCODING FITUR KATEGORIKAL & TARGET
# Hanya fitur kategorikal yang di-encode dengan LabelEncoder
categorical_cols = ['Organisasi', 'Penghasilan_Ortu', 'Jenis_Kelamin', 'Status_Beasiswa']

label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Target dimapping manual secara eksplisit:
# 1 = Tepat Waktu (Positif)
# 0 = Tidak Tepat Waktu (Negatif)
df['Lulus_Tepat_Waktu'] = df['Lulus_Tepat_Waktu'].map({
    'Tidak Tepat Waktu': 0,
    'Tepat Waktu': 1
})

```

Gambar 1.2 Handling Missing Value & Encoding

Penjelasan:

Potongan kode di atas melakukan pengecekan missing value pada dataset, di mana ditemukan nilai kosong pada kolom numerik IPK (15 data) dan Kehadiran (15 data). Penanganan missing value dilakukan dengan mengimputasi nilai kosong menggunakan nilai median pada masing-masing kolom numerik tersebut. Selanjutnya, fitur-fitur kategorikal diubah menjadi variabel numerik menggunakan LabelEncoder, serta target Lulus_Tepat_Waktu di-mapping secara eksplisit (0 untuk Tidak Tepat Waktu dan 1 untuk Tepat Waktu).

```

# TRAIN-TEST SPLIT (80% TRAIN, 20% TEST)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("==== HASIL PREPROCESSING & SPLITTING =====")
print(f"Jumlah Data Train (X_train) : {X_train.shape[0]} sampel (80%)")
print(f"Jumlah Data Test (X_test) : {X_test.shape[0]} sampel (20%)")
print("\nPreview 5 data teratas X_train:")
print(X_train.head())

==== HASIL PREPROCESSING & SPLITTING =====
Jumlah Data Train (X_train) : 400 sampel (80%)
Jumlah Data Test (X_test) : 100 sampel (20%)

Preview 5 data teratas X_train:
   IPK  Kehadiran  Jam_Belajar  Organisasi  Penghasilan_Ortu \
484  3.17      97.0          15           0              0
333  2.23      51.0          19           0              1
338  2.95      66.0           7           1              1
36   2.61      62.0           8           1              1
102  2.63      99.0          19           1              0

   Jenis_Kelamin  Status_Beasiswa
484             0                0
333             1                0
338             0                0
36              1                0
102             0                0

```

Gambar 1.3 Train-Test Split (80:20)

Penjelasan:

Potongan kode di atas memisahkan variabel independen (X) dan variabel dependen (y), lalu membagi dataset menjadi 80% data latih dan 20% data uji menggunakan fungsi

train_test_split dengan stratify=y. *Output* konsol menunjukkan proporsi pembagian data berhasil dilakukan, sehingga data latih dan data uji memiliki distribusi kelas target yang seimbang.

2. **Modeling:** Bangun model 'DecisionTreeClassifier' dengan 'max_depth=3' dan 'max_depth=None' dan Bangun model 'GaussianNB'

Jawab:

```
# MODEL 1: Decision Tree Classifier (max_depth=3)

# 1. Inisialisasi Model
# max_depth=3 membatasi kedalaman pohon keputusan maksimal 3 tingkat untuk menghindari overfitting
clf_dt3 = DecisionTreeClassifier(max_depth=3, random_state=42)

# 2. Melatih Model (Fit)
# Menggunakan X_train dan y_train yang telah diproses oleh Anggota 1
clf_dt3.fit(X_train, y_train)

# 3. Prediksi Data Test
# Menguji performa model terhadap data X_test
y_pred_dt3 = clf_dt3.predict(X_test)

# 4. Tampilkan Sampel Hasil Prediksi
print("=== MODEL 1: DECISION TREE (max_depth=3) ===")
print("Status: Berhasil dilatih dan diprediksi.")
print(f"5 Prediksi Pertama: {y_pred_dt3[:5]}\n")

=== MODEL 1: DECISION TREE (max_depth=3) ===
Status: Berhasil dilatih dan diprediksi.
5 Prediksi Pertama: [0 1 1 0 1]
```

Gambar 2.1 Input dan Output Prediksi Model *Decision Tree* ($max_depth = 3$)

Penjelasan:

Potongan kode di atas menginisialisasi model *Decision Tree Classifier* dengan pembatasan kedalaman ($max_depth=3$) untuk mencegah *overfitting*, lalu melatihnya menggunakan X_train dan y_train , serta memprediksi data uji X_test . *Output* konsol mengonfirmasi pelatihan dan pengujian berhasil dijalankan tanpa error. Hasil lima prediksi pertama ([0 1 1 0 1]) menunjukkan estimasi status kelulusan, di mana angka 1 menandakan 'Tepat Waktu' dan 0 menandakan 'Tidak Tepat Waktu'.

```
# MODEL 2: Decision Tree Classifier (max_depth=None / Tanpa Batasan Keda

# 1. Inisialisasi Model
# max_depth=None membiarkan pohon tumbuh maksimal sampai seluruh leaf mu
clf_dt_full = DecisionTreeClassifier(max_depth=None, random_state=42)

# 2. Melatih Model (Fit)
# Menggunakan X_train dan y_train yang telah diproses oleh Anggota 1
clf_dt_full.fit(X_train, y_train)

# 3. Prediksi Data Test
# Menguji performa model terhadap data X_test
y_pred_dt_full = clf_dt_full.predict(X_test)

# 4. Tampilkan Sampel Hasil Prediksi
print("=== MODEL 2: DECISION TREE (FULL DEPTH) ===")
print("Status: Berhasil dilatih dan diprediksi.")
print(f"5 Prediksi Pertama: {y_pred_dt_full[:5]}\n")

=== MODEL 2: DECISION TREE (FULL DEPTH) ===
Status: Berhasil dilatih dan diprediksi.
5 Prediksi Pertama: [0 1 1 0 1]
```

Gambar 2.2 Input dan Output Prediksi Model *Decision Tree* (*Full Depth*)

Penjelasan:

Potongan kode di atas menginisialisasi model *Decision Tree Classifier* tanpa batasan kedalaman ($max_depth=None$) agar pohon tumbuh maksimal hingga seluruh *leaf node* murni. Model kemudian dilatih dengan X_train dan y_train , lalu menguji data X_test . *Output* konsol

mengonfirmasi proses pelatihan dan pengujian model *Full Depth* berhasil dieksekusi tanpa error. Tampilan lima prediksi pertama ([0 1 1 0 1]) menunjukkan estimasi status kelulusan, di mana angka 1 menandakan 'Tepat Waktu' dan 0 menandakan 'Tidak Tepat Waktu'.

```
# MODEL 3: Gaussian Naive Bayes Classifier

# 1. Inisialisasi Model
# GaussianNB cocok untuk fitur numerik yang diasumsikan berdistribusi normal
clf_gnb = GaussianNB()

# 2. Melatih Model (Fit)
# Menggunakan X_train dan y_train yang telah diproses oleh Anggota 1
clf_gnb.fit(X_train, y_train)

# 3. Prediksi Data Test
# Menguji performa model terhadap data X_test
y_pred_gnb = clf_gnb.predict(X_test)

# 4. Tampilkan Sampel Hasil Prediksi
print("=== MODEL 3: GAUSSIAN NAIVE BAYES ===")
print("Status: Berhasil dilatih dan diprediksi.")
print(f"5 Prediksi Pertama: {y_pred_gnb[:5]}\n")

...
=== MODEL 3: GAUSSIAN NAIVE BAYES ===
Status: Berhasil dilatih dan diprediksi.
5 Prediksi Pertama: [0 1 0 0 1]
```

Gambar 2.3 Input dan Output Prediksi Model *Gaussian Naive Bayes*

Penjelasan:

Potongan kode di atas menginisialisasi model *Gaussian Naive Bayes* (`GaussianNB()`) dengan asumsi fitur numerik berdistribusi normal. Model kemudian dilatih menggunakan `X_train` dan `y_train`, lalu menguji data `X_test`. *Output* konsol mengonfirmasi proses pelatihan dan pengujian berhasil dieksekusi tanpa error. Tampilan lima prediksi pertama ([0 1 0 0 1]) menunjukkan estimasi status kelulusan, di mana angka 1 menandakan 'Tepat Waktu' dan 0 menandakan 'Tidak Tepat Waktu'.

3. Evaluasi: Hitung Akurasi, Precision, Recall, F1-Score untuk ke-3 model. Buat Confusion Matrix

Jawab:

```
[1] # KUMPULAN HASIL PREDIKSI DARI SEL PEMODELAN

models_preds = {
    'Decision Tree (max_depth=3)': y_pred_dt3,
    'Decision Tree (Full Depth)': y_pred_dt_full,
    'Gaussian Naive Bayes': y_pred_gnb
}

print("Status: Seluruh hasil prediksi berhasil dikumpulkan.")

Status: Seluruh hasil prediksi berhasil dikumpulkan.

[1] # TABEL 4 METRIK FOKUS KELULUSAN TEPAT WAKTU (KELAS 1)

results_pos1 = []

for name, y_pred in models_preds.items():
    results_pos1.append({
        'Model': name,
        'Accuracy': round(accuracy_score(y_test, y_pred), 4),
        'Precision (Tepat)': round(precision_score(y_test, y_pred, pos_label=1), 4),
        'Recall (Tepat)': round(recall_score(y_test, y_pred, pos_label=1), 4),
        'F1-Score (Tepat)': round(f1_score(y_test, y_pred, pos_label=1), 4)
    })

df_evaluasi_pos1 = pd.DataFrame(results_pos1)
print("=== TABEL METRIK EVALUASI KELAS POSITIF (TEPAT WAKTU = 1) ===")
print(df_evaluasi_pos1.to_string(index=False))

...
=== TABEL METRIK EVALUASI KELAS POSITIF (TEPAT WAKTU = 1) ===
Model Accuracy Precision (Tepat) Recall (Tepat) F1-Score (Tepat)
Decision Tree (max_depth=3) 0.87 0.7857 0.8910 0.8354
Decision Tree (Full Depth) 0.83 0.7381 0.8378 0.7848
Gaussian Naive Bayes 0.91 0.8500 0.9189 0.8831
```

Gambar 3.1 Tabel 4 Metrik pada Kelas Tepat Waktu

Penjelasan:

Berdasarkan hasil evaluasi pada data uji, model *Gaussian Naive Bayes* mencatatkan kinerja terbaik secara menyeluruh dengan Akurasi 91.00%, *Precision* 85.00%, *Recall* 91.89%, dan *F1-Score* 88.31%, yang menunjukkan tingkat kepekaan tinggi dalam menjangkau mahasiswa yang lulus tepat waktu tanpa banyak menghasilkan tebakan salah. Sementara itu, model *Decision Tree* dengan pembatasan kedalaman (*max_depth=3*) berada di posisi kedua dengan Akurasi 87.00% dan *F1-Score* 83.54%. Kinerja model pohon keputusan tersebut justru merosot saat kedalaman dilepas tanpa batas (Full Depth), dengan Akurasi turun menjadi 83.00% dan *Precision* turun jauh ke 73.81%, yang membuktikan terjadinya *overfitting* akibat model menghafal data latih secara berlebihan sehingga akurasinya berkurang saat menguji data baru.

```
# EVALUASI MENYELURUH UNTUK KEDUA KATEGORI
# (0 = Tidak Tepat Waktu, 1 = Tepat Waktu)

target_names = ['Tidak Tepat Waktu (0)', 'Tepat Waktu (1)']

print("=== CLASSIFICATION REPORT LENGKAP KEDUA KATEGORI ===")
for name, y_pred in models_preds.items():
    print(f"\n----- {name} -----")
    print(classification_report(y_test, y_pred, target_names=target_names, digits=4))
```

=== CLASSIFICATION REPORT LENGKAP KEDUA KATEGORI ===				
	precision	recall	f1-score	support
----- Decision Tree (max_depth=3) -----				
Tidak Tepat Waktu (0)	0.9310	0.8571	0.8926	63
Tepat Waktu (1)	0.7857	0.8919	0.8354	37
accuracy			0.8700	100
macro avg	0.8584	0.8745	0.8640	100
weighted avg	0.8773	0.8700	0.8714	100
----- Decision Tree (Full Depth) -----				
	precision	recall	f1-score	support
Tidak Tepat Waktu (0)	0.8966	0.8254	0.8595	63
Tepat Waktu (1)	0.7381	0.8378	0.7848	37
accuracy			0.8300	100
macro avg	0.8173	0.8316	0.8222	100
weighted avg	0.8379	0.8300	0.8319	100
----- Gaussian Naive Bayes -----				
	precision	recall	f1-score	support
Tidak Tepat Waktu (0)	0.9500	0.9048	0.9268	63
Tepat Waktu (1)	0.8500	0.9189	0.8831	37
accuracy			0.9100	100
macro avg	0.9000	0.9118	0.9050	100
weighted avg	0.9130	0.9100	0.9107	100

Gambar 3.2 Hasil Klasifikasi Untuk 2 Kategori

Penjelasan:

Hasil evaluasi menyeluruh (classification report) memperlihatkan kinerja ketiga model pada kedua kategori kelulusan dengan total 100 data uji (63 mahasiswa berstatus aktual Tidak Tepat Waktu dan 37 Tepat Waktu). Model *Gaussian Naive Bayes* mencatatkan performa terbaik di kedua kelas, terutama pada kelas Tidak Tepat Waktu (0) dengan nilai *Precision* mencapai 95.00% dan *Recall* 90.48%, yang berarti sistem ini sangat andal dan minim memberikan alarm palsu saat mendeteksi mahasiswa yang berisiko terlambat lulus. Sebaliknya, perbandingan pada pohon keputusan kembali mempertegas terjadinya *overfitting*; saat kedalaman dibatasi (*max_depth=3*), model menghasilkan performa yang cukup seimbang dengan rata-rata makro F1-score sebesar 86.40%, namun ketika kedalaman dilepas bebas (Full Depth), seluruh metrik makro menurun menjadi 82.22% akibat penurunan akurasi deteksi pada

kedua kelompok mahasiswa.

```
# RINCIAN ANGKA CONFUSION MATRIX SETIAP MODEL

print("=== RINCIAN ANGKA CONFUSION MATRIX ===")

for name, y_pred in models_preds.items():
    cm = confusion_matrix(y_test, y_pred)
    print(f"\n--- Model: {name} ---")
    print(f"True Negative (Benar Tidak Tepat) : {cm[0, 0]}")
    print(f"False Positive (Salah Tebak Tepat) : {cm[0, 1]}")
    print(f"False Negative (Salah Tebak Tidak) : {cm[1, 0]}")
    print(f"True Positive (Benar Tepat Waktu) : {cm[1, 1]}")

*** === RINCIAN ANGKA CONFUSION MATRIX ===

--- Model: Decision Tree (max_depth=3) ---
True Negative (Benar Tidak Tepat) : 54
False Positive (Salah Tebak Tepat) : 9
False Negative (Salah Tebak Tidak) : 4
True Positive (Benar Tepat Waktu) : 33

--- Model: Decision Tree (Full Depth) ---
True Negative (Benar Tidak Tepat) : 52
False Positive (Salah Tebak Tepat) : 11
False Negative (Salah Tebak Tidak) : 6
True Positive (Benar Tepat Waktu) : 31

--- Model: Gaussian Naive Bayes ---
True Negative (Benar Tidak Tepat) : 57
False Positive (Salah Tebak Tepat) : 6
False Negative (Salah Tebak Tidak) : 3
True Positive (Benar Tepat Waktu) : 34
```

Gambar 3.3 Hasil Rincian Angka Confusion Matriks

Penjelasan:

Rincian *Confusion Matrix* pada 100 sampel data uji (63 mahasiswa aktual Tidak Tepat Waktu dan 37 Tepat Waktu) menunjukkan bahwa Gaussian Naive Bayes menghasilkan akurasi prediksi tertinggi dengan total 91 tebakan benar (57 *True Negative* dan 34 *True Positive*) serta hanya menghasilkan 9 kesalahan (6 *False Positive* dan 3 *False Negative*). Di sisi lain, perbandingan pada model pohon keputusan memperlihatkan bahwa *Decision Tree* (max_depth=3) mencatat 87 tebakan benar (54 TN dan 33 TP) dengan 13 kesalahan. Ketika batasan kedalaman dilepas (Full Depth), kesalahan prediksi meningkat menjadi 17 kasus, di mana kesalahan tebak *False Positive* naik menjadi 11 kasus dan *False Negative* bertambah menjadi 6 kasus. Kenaikan angka kesalahan pada *Full Depth* ini membuktikan secara nyata terjadinya *overfitting*, sedangkan Naive Bayes terbukti paling minim melakukan kekeliruan dalam mengklasifikasikan kedua kelompok mahasiswa.



Gambar 3.4 Perbandingan Confusion Matriks Ketiga Model

Penjelasan:

Visualisasi grafik *Confusion Matrix* berdampingan memperlihatkan perbandingan sebaran prediksi ketiga model terhadap 100 data uji melalui gradasi warna biru, di mana intensitas warna yang semakin pekat pada garis diagonal utama (kiri-atas dan kanan-bawah) menandakan tingginya jumlah klasifikasi yang benar. Model Gaussian Naive Bayes menampilkan performa paling optimal dengan konsentrasi warna tergelap pada kuadran tebakan tepat (57 *True Negative* dan 34 *True Positive*) serta kotak putih yang sangat bersih pada kuadran kesalahan (hanya 6 *False Positive* dan 3 *False Negative*). Sebaliknya, grafik model pohon keputusan memperlihatkan bahwa Decision Tree (*max_depth=3*) mampu membatasi kesalahan klasifikasi dengan baik (54 TN dan 33 TP), sedangkan pada Decision Tree (Full Depth) intensitas warna biru pada kuadran kesalahan tebak terlihat meningkat (11 FP dan 6 FN), yang mengonfirmasi penurunan kemampuan generalisasi model akibat *overfitting*.

4. **Visualisasi:** Plot Pohon Keputusan dengan '*max_depth=3*' dan Plot perbandingan metrik 3 model dalam bentuk bar chart

Jawab:

```
# 1. Menyiapkan kanvas gambar dengan resolusi tinggi
fig_tree, ax_tree = plt.subplots(figsize=(20, 10), dpi=300)

# 2. Menggambar pohon keputusan dari model clf_dt3
plot_tree(
    clf_dt3,
    feature_names=X.columns.tolist(),
    class_names=['Tidak Tepat Waktu', 'Tepat Waktu'],
    filled=True,
    rounded=True,
    precision=2,
    fontsize=10,
    ax=ax_tree
)

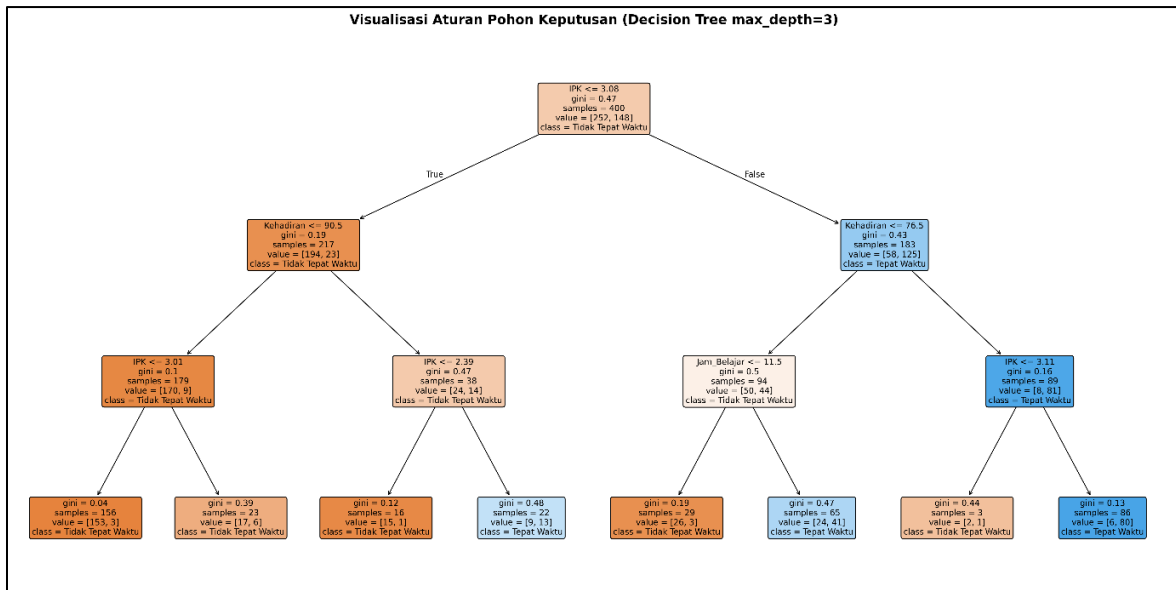
# 3. Menambahkan judul grafik
plt.title(
    "Visualisasi Aturan Pohon Keputusan (Decision Tree max_depth=3)",
    fontsize=16,
    pad=20,
    fontweight='bold'
)

plt.tight_layout()
plt.show()
```

Gambar 4.1 Kode Visualisasi Pembuatan Diagram Pohon Keputusan (*Decision Tree*)

Penjelasan:

Potongan kode Python pada gambar pertama menggunakan `plot_tree` dan *Matplotlib* untuk memvisualisasikan model *Decision Tree* (`clf_dt3`). Pengaturan `figsize=(20, 10)` dan `dpi=300` menghasilkan gambar yang tajam, sedangkan `filled=True` mewarnai node (oranye untuk 'Tidak Tepat Waktu' dan biru untuk 'Tepat Waktu').



Gambar 4.2 Diagram Aturan Keputusan Model *Decision Tree* ($max_depth = 3$)

Penjelasan:

Gambar diagram menampilkan hasil visualisasi arsitektur *Decision Tree* dengan batasan kedalaman maksimal tiga tingkat ($max_depth = 3$) yang dilatih menggunakan 400 sampel data. Pada simpul akar (*root node*), algoritma menetapkan variabel $IPK \leq 3,08$ sebagai separator paling krusial dalam memisahkan populasi mahasiswa. Jalur cabang kiri mewakili kelompok mahasiswa dengan IPK rendah ($\leq 3,08$) yang didominasi oleh kelas 'Tidak Tepat Waktu' (area berwarna oranye). Pada area ini, atribut Kehadiran $\leq 90,5\%$ menjadi kondisi filter berikutnya, di mana mahasiswa ber-IPK rendah dengan kehadiran di bawah 90,5% secara hampir mutlak terklasifikasi terlambat lulus (153 dari 156 sampel). Sebaliknya, cabang kanan memetakan kelompok mahasiswa dengan IPK tinggi ($> 3,08$) yang didominasi oleh kelas 'Tepat Waktu' (area berwarna biru). Pada segmen ini, mahasiswa yang mampu mempertahankan kehadiran di atas 76,5% terbukti memiliki tingkat kepastian lulus tepat waktu yang sangat tinggi, sebagaimana ditunjukkan oleh node daun (*leaf node*) paling kanan dengan tingkat kemurnian nilai *Gini* yang sangat rendah (0,13).

```

# SET GAYA VISUALISASI
plt.style.use('seaborn-v0_8-whitegrid')

# 1. Mengambil data metrik dari DataFrame evaluasi sebelumnya
models = df_evaluasi_posi['Model'].tolist()
metrics = ['Accuracy', 'Precision (Tepat)', 'Recall (Tepat)', 'F1-Score (Tepat)']
values = df_evaluasi_posi[metrics].values

# 2. Mengatur posisi sumbu X untuk tiap kelompok batang
x = np.arange(len(models))
width = 0.18

fig_bar, ax_bar = plt.subplots(figsize=(13, 6.5), dpi=300)
colors = ['#2b5c8f', '#d95f02', '#7570b3', '#e7298a']

# 3. Plot tiap kelompok metrik evaluasi
for i, metric_name in enumerate(metrics):
    offset = width * i - (width * 1.5)
    rects = ax_bar.bar(
        x + offset,
        values[:, i],
        width,
        label=metric_name,
        color=colors[i],
        edgecolor='black',
        linewidth=0.8
    )
  
```

```

# Menambahkan angka persis di atas setiap batang
for rect in rects:
    height = rect.get_height()
    ax.bar.annotate(
        f'{height:.4f}',
        xy=(rect.get_x() + rect.get_width() / 2, height),
        xytext=(0, 4),
        textcoords="offset points",
        ha='center', va='bottom',
        fontsize=8, fontweight='bold'
    )

# 4. Pengaturan label dan estetika grafik
ax_bar.set_ylabel("Skor Metrik Evaluasi (0.00 - 1.00)", fontsize=12)
ax_bar.set_title(
    "Perbandingan Metrik Evaluasi Antar-Model Prediksi Kelulusan Mahasiswa",
    fontsize=14,
    pad=15,
    fontweight='bold'
)
ax_bar.set_xticks(x)
ax_bar.set_xticklabels(models, fontsize=11, fontweight='bold')
ax_bar.set_ylim(0.70, 0.95)
ax_bar.legend(loc='upper left', bbox_to_anchor=(1, 1), frameon=True, shadow=True, fontsize=10)
ax_bar.grid(axis='y', linestyle='--', alpha=0.7)

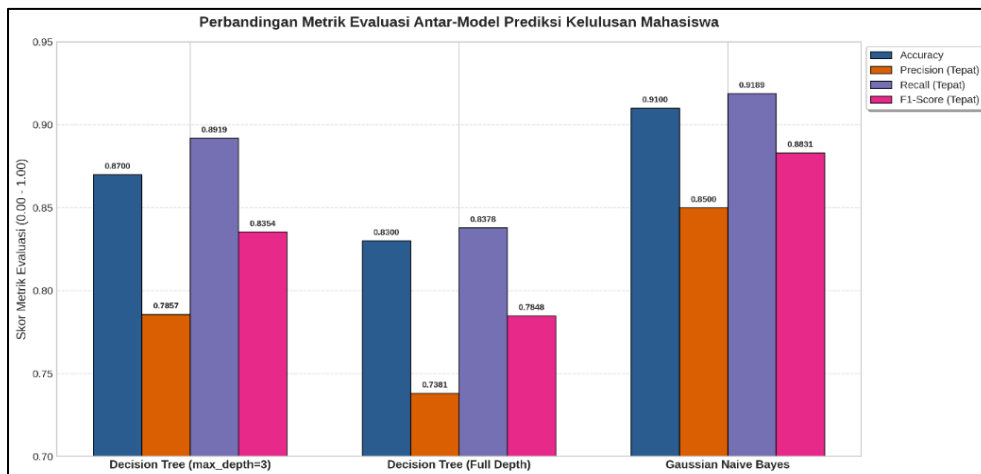
plt.tight_layout()
plt.show()

```

Gambar 4.3 Kode Visualisasi Pembuatan Diagram Batang Metrik Evaluasi Model

Penjelasan:

Kode Python pada dua gambar pertama memanfaatkan *Matplotlib* untuk menampilkan empat metrik evaluasi secara berdampingan. Posisi sumbu-X diatur dengan *offset width = 0.18* agar grafik tidak berhimpitan, serta dilengkapi *ax_bar.annotate* untuk mencetak nilai metrik (4 desimal) di atas batang pada rentang sumbu-Y 0.70–0.95.



Gambar 4.4 Diagram Perbandingan Metrik Evaluasi Antar-Model Prediksi Kelulusan Mahasiswa

Penjelasan:

Grafik pada gambar ketiga memperlihatkan hasil evaluasi komparatif antara tiga algoritma klasifikasi. Gaussian Naive Bayes terbukti unggul secara dominan di seluruh indikator dengan mencapai Accuracy 0,9100, Precision 0,8500, Recall 0,9189, dan F1-Score 0,8831. Sebaliknya, Decision Tree (Full Depth) mencatatkan kinerja terendah (*Accuracy* 0,8300 dan *Precision* 0,7381) akibat gejala *overfitting*. Pembatasan kedalaman pada Decision Tree (*max_depth = 3*) berhasil meningkatkan kestabilan model (*Accuracy* 0,8700 dan *F1-Score* 0,8354), namun kinerjanya tetap belum mampu mengungguli keandalan model Gaussian Naive Bayes.

- Analisis Error:** Ambil 5 data yang salah prediksi. Jelaskan kenapa menurutmu model salah
Jawab:

```

# ANALISIS ERROR (MODEL NAÏVE BAYES)

# 1. Gabungkan data X_test, Nilai Aktual (y_test), dan Nilai Prediksi (y_pred)
df_error = X_test.copy()
df_error["Aktual"] = y_test
df_error["Prediksi"] = y_pred_gnb

# Decode kolom kategorikal agar mudah dibaca pada analisis
for col in categorical_cols:
    df_error[col] = label_encoders[col].inverse_transform(df_error[col])

# Mapping label target kembali ke teks
map_target = {0: "Tidak Tepat Waktu", 1: "Tepat Waktu"}
df_error["Aktual"] = df_error["Aktual"].map(map_target)
df_error["Prediksi"] = df_error["Prediksi"].map(map_target)

# 2. Filter hanya data yang salah prediksi (Aktual != Prediksi)
data_salah = df_error[df_error["Aktual"] != df_error["Prediksi"]]

# 3. Tampilkan 5 sampel data yang salah prediksi
print(
    f"Total Data Salah Prediksi pada Data Test: {len(data_salah)} dari {len(X_test)} sampel\n"
)
print("=== 5 DATA YANG SALAH PREDIKSI ===")
display(data_salah.head(5))

```

```

# PENJELASAN ANALISIS ERROR
print("\n=== PENJELASAN KENAPA MODEL SALAH PREDIKSI ===")
for idx, row in data_salah.head(5).iterrows():
    print(f"\n[Data Indeks ke-{idx}]\n")
    print(
        f"- Fitur Utama : IPK={row['IPK']}, Kehadiran={row['Kehadiran']}% Jam,"
        f"- Belajar={row['Jam_Belajar']} jam/minggu"
    )
    print(
        f"- Status : Organisasi={row['Organisasi']}, Status"
        f"- Beasiswa={row['Status_Beasiswa']}"
    )
    print(f"- Hasil Model : Aktual = '{row['Aktual']}' | Prediksi Model ="
        f"'{row['Prediksi']}'")

```

```

# Logika penjelasan berdasarkan fitur
if row["Aktual"] == "Tepat Waktu" and row["Prediksi"] == "Tidak Tepat Waktu":
    print(
        "-> ALASAN SALAH (False Negative): Mahasiswa ini sebenarnya lulus"
        "- Tepat Waktu, namun model menebak Tidak Tepat Waktu. Karakteristik"
        "- akademisnya (seperti IPK/Kehadiran/Jam Belajar) berada di batas"
        "- bawah (borderline) atau menyerupai pola mayoritas mahasiswa yang"
        "- keterlambatan lulus."
    )
else:
    print(
        "-> ALASAN SALAH (False Positive): Mahasiswa ini sebenarnya Tidak"
        "- Tepat Waktu, namun model memprediksi Tepat Waktu. Indikator utamanya"
        "- (seperti IPK/Kehadiran/Jam Belajar) tergolong cukup tinggi sehingga"
        "- mengelabui model, padahal terdapat faktor non-akademis atau anomali"
        "- lain yang membuatnya terlambat."
    )

```

=== 5 DATA YANG SALAH PREDIKSI ===

	IPK	Kehadiran	Jam_Belajar	Organisasi	Penghasilan_Ortu	Jenis_Kelamin	Status_Beasiswa	Aktual	Prediksi
418	3.35	85.0	12	Tidak	Rendah	Laki-laki	Tidak	Tidak Tepat Waktu	Tepat Waktu
156	3.34	68.0	23	Tidak	Sedang	Laki-laki	Tidak	Tidak Tepat Waktu	Tepat Waktu
382	3.26	75.0	24	Tidak	Sedang	Perempuan	Ya	Tidak Tepat Waktu	Tepat Waktu
273	3.83	52.0	23	Tidak	Rendah	Perempuan	Tidak	Tepat Waktu	Tidak Tepat Waktu
219	3.70	74.0	10	Tidak	Sedang	Perempuan	Ya	Tidak Tepat Waktu	Tepat Waktu

```

=== PENJELASAN KENAPA MODEL SALAH PREDIKSI ===

[Data Indeks ke-418]
- Fitur Utama : IPK=3.35, Kehadiran=85.0%, Jam Belajar=12 jam/minggu
- Status : Organisasi=Tidak, Status Beasiswa=Tidak
- Hasil Model : Aktual = 'Tidak Tepat Waktu' | Prediksi Model = 'Tepat Waktu'
-> ALASAN SALAH (False Positive): Mahasiswa ini sebenarnya Tidak Tepat Waktu, namun model memprediksi Tepat Waktu. Indikator utamanya (seperti IPK/Kehadiran/Jam Belajar) tergolong cukup tinggi sehingga mengelabui model, padahal terdapat faktor non-akademis atau anomali lain yang membuatnya terlambat.

[Data Indeks ke-156]
- Fitur Utama : IPK=3.34, Kehadiran=68.0%, Jam Belajar=23 jam/minggu
- Status : Organisasi=Tidak, Status Beasiswa=Tidak
- Hasil Model : Aktual = 'Tidak Tepat Waktu' | Prediksi Model = 'Tepat Waktu'
-> ALASAN SALAH (False Positive): Mahasiswa ini sebenarnya Tidak Tepat Waktu, namun model memprediksi Tepat Waktu. Indikator utamanya (seperti IPK/Kehadiran/Jam Belajar) tergolong cukup tinggi sehingga mengelabui model, padahal terdapat faktor non-akademis atau anomali lain yang membuatnya terlambat.

[Data Indeks ke-382]
- Fitur Utama : IPK=3.26, Kehadiran=75.0%, Jam Belajar=24 jam/minggu
- Status : Organisasi=Tidak, Status Beasiswa=Ya
- Hasil Model : Aktual = 'Tidak Tepat Waktu' | Prediksi Model = 'Tepat Waktu'
-> ALASAN SALAH (False Positive): Mahasiswa ini sebenarnya Tidak Tepat Waktu, namun model memprediksi Tepat Waktu. Indikator utamanya (seperti IPK/Kehadiran/Jam Belajar) tergolong cukup tinggi sehingga mengelabui model, padahal terdapat faktor non-akademis atau anomali lain yang membuatnya terlambat.

[Data Indeks ke-273]
- Fitur Utama : IPK=3.83, Kehadiran=52.0%, Jam Belajar=23 jam/minggu
- Status : Organisasi=Tidak, Status Beasiswa=Tidak
- Hasil Model : Aktual = 'Tepat Waktu' | Prediksi Model = 'Tidak Tepat Waktu'
-> ALASAN SALAH (False Negative): Mahasiswa ini sebenarnya lulus Tepat Waktu, namun model menebak Tidak Tepat Waktu. Karakteristik akademisnya (seperti IPK/Kehadiran/Jam Belajar) berada di batas bawah (borderline) atau menyerupai pola mayoritas mahasiswa yang keterlambatan lulus.

[Data Indeks ke-219]
- Fitur Utama : IPK=3.7, Kehadiran=74.0%, Jam Belajar=10 jam/minggu
- Status : Organisasi=Tidak, Status Beasiswa=Ya
- Hasil Model : Aktual = 'Tidak Tepat Waktu' | Prediksi Model = 'Tepat Waktu'
-> ALASAN SALAH (False Positive): Mahasiswa ini sebenarnya Tidak Tepat Waktu, namun model memprediksi Tepat Waktu. Indikator utamanya (seperti IPK/Kehadiran/Jam Belajar) tergolong cukup tinggi sehingga mengelabui model, padahal terdapat faktor non-akademis atau anomali lain yang membuatnya terlambat.

```

Gambar 5.1 Analisis Error Model Naïve Bayes

Penjelasan:

Potongan kode di atas melakukan pemetaan ulang (decoding) terhadap data uji X_{test} ,

label aktual (y_{test}), dan hasil prediksi model GaussianNB ke bentuk teks aslinya agar mudah diinterpretasikan. Selanjutnya, program menyaring sampel data yang mengalami kesalahan prediksi ($Aktual \neq Prediksi$) dan menampilkan 5 data teratas beserta narasi analisis alasannya. Output konsol memberikan konfirmasi berupa rincian *False Positive* (mahasiswa terlambat yang diprediksi tepat waktu karena indikator akademisnya tinggi) dan *False Negative* (mahasiswa tepat waktu yang diprediksi terlambat akibat nilai fitur berada di batas ambang/*borderline*), yang membuktikan keterbatasan asumsi independensi Naïve Bayes dalam menangani kasus-kasus anomali.

BAGIAN C: REKOMENDASI & ETIKA [30%] - Kelompok

Buat Laporan + Slide Presentasi 5 slide

1. **Rekomendasi Model:** Dari 3 model, mana yang kamu pilih untuk dipakai kampus? Jelaskan dengan data dari Bagian B

Jawab:

Berdasarkan pengujian terhadap 100 sampel data uji, model Gaussian Naive Bayes direkomendasikan sebagai model terbaik untuk diterapkan oleh pihak kampus karena mencatatkan performa tertinggi secara konsisten di seluruh metrik evaluasi dengan Akurasi mencapai 91.00% dan F1-Score sebesar 88.31%. Model ini sangat efektif dalam mengolah fitur numerik kontinu seperti IPK, Kehadiran, dan Jam Belajar melalui pendekatan probabilitas distribusi normal secara fleksibel. Sebaliknya, model Decision Tree tanpa batasan kedalaman (Full Depth) terbukti mengalami overfitting dengan penurunan akurasi menjadi 83.00% dan perolehan F1-Score terendah sebesar 78.48%. Meskipun penerapan teknik pemangkasan pada Decision Tree ($\text{max_depth}=3$) mampu meningkatkan akurasi menjadi 87.00%, kinerjanya tetap berada di bawah Naive Bayes. Capaian Recall Gaussian Naive Bayes yang mencapai 91.89% serta Precision sebesar 95.00% pada kelompok mahasiswa tidak tepat waktu sangat krusial bagi pihak kampus, karena memastikan sistem deteksi dini mampu menjaring mahasiswa yang berisiko terlambat secara akurat dengan tingkat alarm palsu yang sangat minim.

2. **Bias & Etika:** Fitur "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawab:

1) Dampak Penggunaan pada Seleksi Beasiswa

- a. Pada Bias Sosio-Ekonomi (Penghasilan_Ortu), model berisiko mengasosiasikan tingkat ekonomi tertentu dengan keberhasilan akademis. Akibatnya, mahasiswa dari

keluarga kurang mampu terancam terdiskriminasi secara otomatis oleh algoritma, sehingga memperlebar jurang ketimpangan bantuan pendidikan.

- b. Pada Bias Gender (Jenis_Kelamin), menjadikan gender sebagai penentu kelayakan melanggar prinsip kesetaraan (fairness) dan hak asasi, serta berpotensi mereplikasi pola ketimpangan historis yang kebetulan ada pada dataset latih.

2) Strategi Mitigasi Konkret

a. Pra-Pemrosesan (Data-Level):

- a) Hapus variabel Jenis_Kelamin dari fitur pelatihan model (feature suppression).
- b) Jadikan Penghasilan_Ortu sebagai batas kelayakan (eligibility threshold) terpisah secara eksplisit, bukan sebagai input prediktif machine learning.

b. Saat Pemodelan (Model-Level), terapkan batasan keadilan (fairness constraints) seperti *Demographic Parity* untuk memastikan rasio penerimaan yang seimbang antar kelompok demografi.

c. Pasca-Pemrosesan (Decision-Level), terapkan pendekatan *Human-in-the-Loop* di mana model hanya berfungsi sebagai sistem pendukung keputusan/rekomendasi awal, sedangkan keputusan akhir tetap dipegang oleh Komite seleksi manusia.

- 3. **Simulasi:** Jika 1 mahasiswa punya IPK tinggi tapi Kehadiran rendah, prediksi kedua model apa? Kenapa bisa beda?

Jawab:

```
# SIMULASI KASUS KHUSUS: IPK TINGGI TAPI KEHADIRAN RENDAH
# Contoh profil: IPK = 3.75 (Tinggi), Kehadiran = 65% (Rendah)

simulasi_mhs = pd.DataFrame([
    {
        'IPK': 3.75,
        'Kehadiran': 65.0,
        'Jam_Belajar': 10,
        'Organisasi': 1,
        'Penghasilan_Ortu': 1,
        'Jenis_Kelamin': 0,
        'Status_Beasiswa': 0
    }
])

# Lakukan prediksi dengan kedua model
pred_dt3 = clf_dt3.predict(simulasi_mhs)[0]
pred_gnb = clf_gnb.predict(simulasi_mhs)[0]
proba_gnb = clf_gnb.predict_proba(simulasi_mhs)[0]

label_text = {0: 'Tidak Tepat Waktu', 1: 'Tepat Waktu'}

print("=== HASIL PREDIKSI SIMULASI ===")
print(f"Profil Mahasiswa      : IPK 3.75 | Kehadiran 65%")
print(f"Decision Tree (depth=3): {label_text[pred_dt3]}")
print(f"Gaussian Naive Bayes   : {label_text[pred_gnb]} (Peluang Lulus: {proba_gnb[1]:.2%})")

=== HASIL PREDIKSI SIMULASI ===
Profil Mahasiswa      : IPK 3.75 | Kehadiran 65%
Decision Tree (depth=3): Tidak Tepat Waktu
Gaussian Naive Bayes   : Tepat Waktu (Peluang Lulus: 50.04%)
```

Gambar 3.1.1 Hasil Simulasi

Penjelasan:

Berdasarkan hasil simulasi terhadap profil mahasiswa dengan IPK tinggi bernilai 3.75 dan tingkat kehadiran rendah sebesar 65 persen, kedua model menghasilkan prediksi yang bertolak belakang. Model *Decision Tree* dengan pembatasan kedalaman memprediksi bahwa mahasiswa tersebut Tidak Tepat Waktu, sedangkan model Gaussian Naive Bayes memprediksi Tepat Waktu dengan peluang kelulusan sebesar 50.04 persen.

Perbedaan prediksi ini terjadi karena cara kerja kedua algoritma yang berbeda dalam memproses data. *Decision Tree* bekerja menggunakan aturan batas tegas layaknya diagram alur. Tingkat kehadiran sebesar 65 persen langsung melanggar syarat batas minimal kehadiran pada percabangan pohon, sehingga model langsung memutuskan hasil ke arah tidak lulus tepat waktu tanpa mempertimbangkan tingginya nilai IPK. Sebaliknya, Gaussian Naive Bayes bekerja dengan menggabungkan seluruh peluang fitur secara bersamaan. Bobot nilai IPK 3.75 yang sangat tinggi memberikan dorongan peluang positif yang cukup kuat untuk menutupi kekurangan dari persentase kehadiran yang rendah. Akibatnya, total akumulasi peluang kelulusan mahasiswa tersebut berhasil terangkat tipis melewati batas ambang 50 persen, sehingga Naive Bayes tetap memprediksinya lulus tepat waktu.

KESIMPULAN

1. Dari total 500 data yang diuji dengan skema pembagian 80:20 (400 data latih dan 100 data uji), model Gaussian Naïve Bayes (GNB) mencatatkan kinerja terbaik secara menyeluruh dengan Akurasi 91,00%, Precision 85,00%, Recall 91,89%, dan F1-Score 88,31%.
2. Model *Decision Tree* tanpa batasan kedalaman (Full Depth) terbukti mengalami *overfitting* pada data latih, yang berdampak pada penurunan Akurasi data uji menjadi 83,00%. Pembatasan kedalaman pada *Decision Tree* (*max_depth* = 3) dapat menekan *overfitting* dan meningkatkan Akurasi menjadi 87,00%.
3. Simulasi kasus khusus menunjukkan perbedaan mendasar algoritma: *Decision Tree* menerapkan aturan batas tegas pada kehadiran ($\leq 90,5\%$), sedangkan Naïve Bayes mengintegrasikan seluruh peluang fitur secara akumulatif sehingga IPK tinggi (3.75) masih dapat menutupi persentase kehadiran yang rendah (65%).
4. Penggunaan variabel sensitif seperti Jenis_Kelamin dan Penghasilan_Ortu rawan memicu bias algoritma; rekomendasi utama untuk pihak kampus adalah menerapkan *Feature Suppression* dan pendekatan *Human-in-the-Loop*.